

File Handling

Q1.

Use FileWriter to create a file named playlist.txt and write the names of 5 songs you listen to on Spotify, each on a new line.

Answer:

```
import java.io.FileWriter;
import java.io.IOException;

public class CreatePlaylistFile {
    public static void main(String[] args) {
        String[] songs = {
            "Blinding Lights",
            "Kesariya",
            "Flowers",
            "Apna Bana Le",
            "Shape of You"
        };

        try (FileWriter writer = new FileWriter("playlist.txt")) {
            for (String song : songs) {
                writer.write(song + "\n");
            }
            System.out.println("playlist.txt created successfully with 5 songs.");
        } catch (IOException e) {
            System.out.println("An error occurred while writing the file: " + e.getMessage());
        }
    }
}

/* Sample Output:
playlist.txt created successfully with 5 songs.
```

Contents of playlist.txt:

```
Blinding Lights
Kesariya
Flowers
Apna Bana Le
Shape of You
*/
```

Q2.

Read the playlist.txt file using FileReader and display each song name in the console with its line number, like '1. Blinding Lights'.

Answer:

```
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;

public class ReadPlaylistFile {
    public static void main(String[] args) {
        try (FileReader fr = new FileReader("playlist.txt");
```

```

        BufferedReader br = new BufferedReader(fr)) {

    String line;
    int lineNumber = 1;

    while ((line = br.readLine()) != null) {
        System.out.println(lineNumber + ". " + line);
        lineNumber++;
    }

    } catch (IOException e) {
        System.out.println("An error occurred while reading the file: " + e.getMessage());
    }
}

/* Sample Output:
1. Blinding Lights
2. Kesariya
3. Flowers
4. Apna Bana Le
5. Shape of You
*/

```

Q3.

Simulate a Flipkart shopping cart log: use `FileOutputStream` to append a line like 'Added: Redmi Note 12 - Rs. 12999' to a file named `cartlog.txt` every time you call a function `addToCart()`.

Answer:

```

import java.io.FileOutputStream;
import java.io.IOException;

public class CartLogger {

    public static void addToCart(String productName, double price) {
        String logLine = "Added: " + productName + " - Rs. " + price + "\n";

        // 'true' enables append mode so previous entries aren't overwritten
        try (FileOutputStream fos = new FileOutputStream("cartlog.txt", true)) {
            fos.write(logLine.getBytes());
            System.out.println("Logged: " + logLine.trim());
        } catch (IOException e) {
            System.out.println("An error occurred while writing to cartlog.txt: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        addToCart("Redmi Note 12", 12999);
        addToCart("Bluetooth Earbuds", 1499);
        addToCart("Laptop Sleeve", 599);
    }
}

/* Sample Output:
Logged: Added: Redmi Note 12 - Rs. 12999.0
Logged: Added: Bluetooth Earbuds - Rs. 1499.0
Logged: Added: Laptop Sleeve - Rs. 599.0

```

Contents of `cartlog.txt` after running:
Added: Redmi Note 12 - Rs. 12999.0

```
Added: Bluetooth Earbuds - Rs. 1499.0
Added: Laptop Sleeve - Rs. 599.0
*/
```

Q4.

Use `FileInputStream` to read the `cartlog.txt` file and count how many items have been added to the cart by counting the number of lines. Hint: Read the file byte by byte, convert to characters, and count newline characters.

Answer:

```
import java.io.FileInputStream;
import java.io.IOException;

public class CartLogCounter {
    public static void main(String[] args) {
        int lineCount = 0;

        try (FileInputStream fis = new FileInputStream("cartlog.txt")) {
            int byteData;
            while ((byteData = fis.read()) != -1) {
                char ch = (char) byteData;
                if (ch == '\n') {
                    lineCount++;
                }
            }
            System.out.println("Total items added to cart: " + lineCount);
        } catch (IOException e) {
            System.out.println("An error occurred while reading cartlog.txt: " + e.getMessage());
        }
    }
}

/* Sample Output:
Total items added to cart: 3
*/
```

Q5.

Explain in 3-4 sentences how saving your playlist or shopping cart logs to a file with `FileWriter/FileOutputStream` is similar to how apps like Zomato or Myntra save your data before they use a real database.

Answer:

Writing playlist or cart data to a plain text file with `FileWriter` or `FileOutputStream` is essentially a simple, persistent storage mechanism, just like a database - the key difference is that a file has no structure, indexing, or query language, so you can only read it back sequentially. Apps like Zomato or Myntra actually do exactly this kind of durable writing under the hood, except they use a proper database (like MySQL or MongoDB) instead of raw text files, which lets them search, filter, and update specific records instantly instead of scanning an entire file. Both approaches share the same core idea: data created during the app's runtime (a cart, a playlist) needs to be saved somewhere permanent, or it disappears the moment the program closes.